

این اصل می گوید:
هر کلاسی که از یک کلاس دیگر (کلاس والد) ارث‌بری می‌کند، باید بتواند به جای کلاس والد بدون ایجاد مشکل استفاده شود.
به بیان ساده: جایگزینی کلاس فرزند با کلاس والد نباید رفتار برنامه را خراب کند

چرا این مهم است؟

:اگر کلاس فرزند نتواند به درستی جایگزین کلاس والد شود
کدهایی که روی کلاس والد نوشته شده‌اند، ممکن است با کلاس فرزند شکست بخورند.
تست و نگهداری سخت می‌شود
اصول OOP و انعطاف‌پذیری نقض می‌شود.

مثال نقض LSP در C#

فرض کنید کلاس والد Rectangle و کلاس فرزند Square داریم:

```
public class Rectangle
{
    public virtual int Width { get; set; }
    public virtual int Height { get; set; }

    public int GetArea()
    {
        return Width * Height;
    }
}

public class Square : Rectangle
{
    public override int Width
    {
        set { base.Width = base.Height = value; }
    }

    public override int Height
    {
        set { base.Width = base.Height = value; }
    }
}
```

```
}  
}  
}
```

مشکل :

رفتار برنامه خراب می‌شود، چون Square قوانین Rectangle را تغییر داده است.

این نقض LSP است.

```
Rectangle rect = new Square();  
rect.Width = 5;  
rect.Height = 10;  
Console.WriteLine($"مساحت: {rect.GetArea()}"); // خروجی واقعی: 100
```

اصلاح مطابق LSP

راه درست: از وراثت اشتباه استفاده نکنیم و از یک اینترفیس یا کلاس پایه انتزاعی استفاده کنیم:

```
public interface IShape  
{  
    int GetArea();  
}  
  
public class RectangleFixed : IShape  
{  
    public int Width { get; set; }  
    public int Height { get; set; }  
  
    public int GetArea()  
    {  
        return Width * Height;  
    }  
}  
  
public class SquareFixed : IShape  
{  
    public int Side { get; set; }  
  
    public int GetArea()  
    {  
        return Side * Side;  
    }  
}
```

حالا:

هر دو Rectangle و Square می‌توانند از IShape استفاده کنند.

هیچ کلاسی مجبور نیست رفتار دیگری را نقض کند.

جایگزینی بدون مشکل انجام می‌شود:

```
IShape shape1 = new RectangleFixed { Width = 5, Height = 10 };
IShape shape2 = new SquareFixed { Side = 5 };

Console.WriteLine($"RectangleFixed مساحت: {shape1.GetArea()}"); // 50
Console.WriteLine($"SquareFixed مساحت: {shape2.GetArea()}"); // 25
```

نتیجه:

وراثت باید منطقی باشد و قوانین کلاس والد را نقض نکند.

اگر کلاس فرزند قوانین والد را تغییر دهد، جایگزینی باعث خرابی می‌شود.

استفاده از اینترفیس یا کلاس پایه انتزاعی، انعطاف بیشتری می‌دهد و LSP را رعایت می‌کند.